RMIT Classification: Trusted

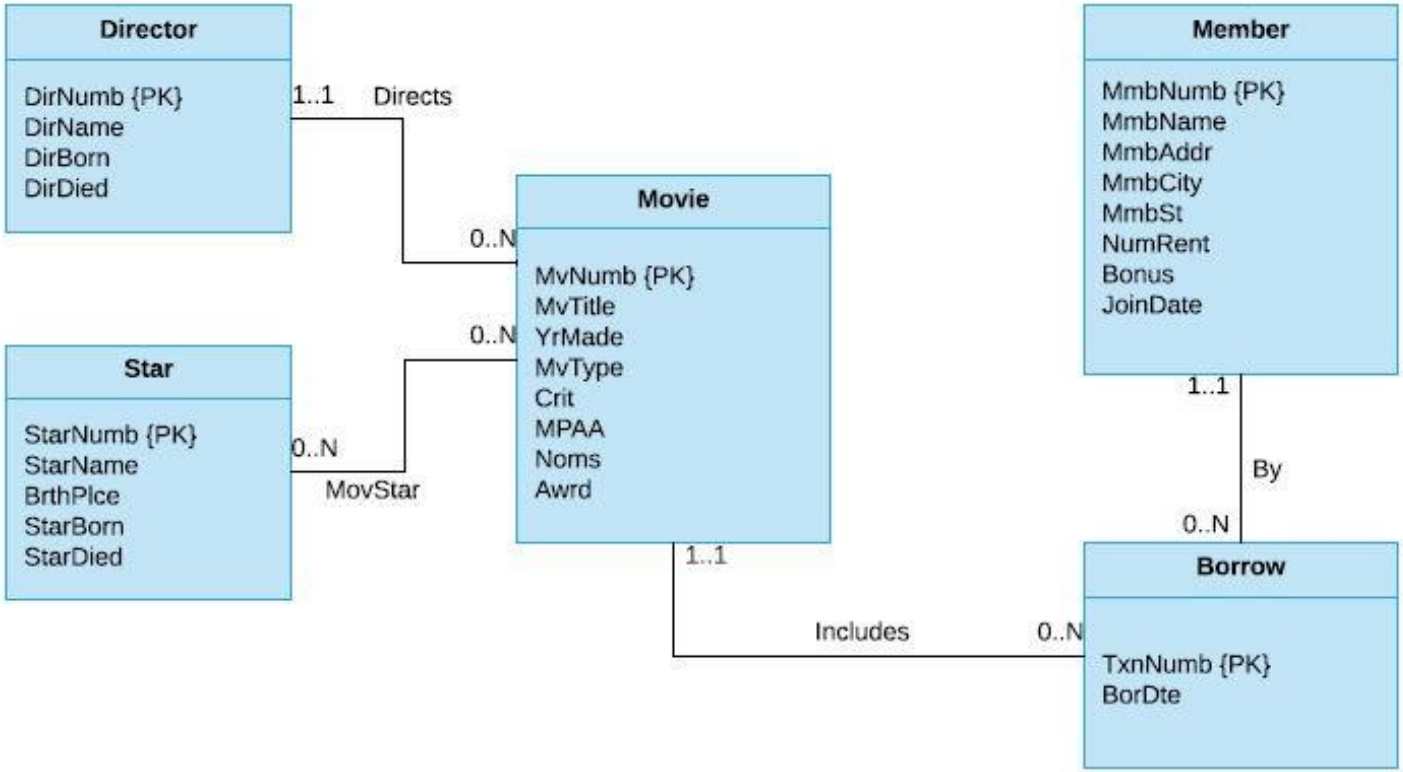
(Practical) Database Concepts (ISYS3412/ISYS1055)

Dr Xiangmin (Emily) Zhou
RMIT University

Lectorial 5: Basic SQL Programming - Part 1

The movies database is built to store information about movies, directors, and stars (i.e. actors and actresses) in a video store. In addition, it stores information on members (who got active memberships to borrow videos from the video store) and their borrowing transactions.

A sketchy ER diagram for the movies database is given below.



SELECT-FROM-WHERE

- An SQL query consists of three components:
  - a SELECT clause,
  - a FROM clause,
  - an optional WHERE clause

SELECT desired attributes
FROM one or more tables
WHERE conditions on tuples of the tables

- Example:

movie(nvnumb, mvtitle, yrmde, mvtype, crit, mpaa, noms, awrd, dirnumb)

```
SELECT mvtitle
FROM movie
WHERE mvtype = 'COMEDY';
```

SELECT Expressions

- Any expression that makes sense can appear as an element of a SELECT clause
- Example: Find the birth century of movie stars.

```
SELECT starname, starborn/100
FROM star;
```

- Attributes can be renamed using AS …. This is often used for producing reports.

```
SELECT starname, starborn/100 AS "CenturyOfBirth"
FROM star;
```

- Select Constants

```
SELECT 'I like ', title
FROM movie
WHERE mvtype = 'COMEDY';
```

The WHERE Clause

- In the WHERE clause, an expression is evaluated to a boolean value (TRUE/FALSE). Tuples are selected only if the WHERE expression is evaluated to TRUE.
- Comparisons result in boolean values

  =, <>, <, >, <=, >=, !=.

  Example: Find the type of movies whose mvID is > 3.

- Many other operators also result in boolean values.
  LIKE, IS NULL, IS NOT NULL
- Logical operators combine boolean expressions into a complex condition (NOT, AND, OR)

  Example:

```
SELECT title
FROM movie
WHERE mvtype='COMEDY'
AND mpaa='PG';
```

String Patterns for Comparison

- A condition can compare a string to a pattern by:

  <Attribute> LIKE <pattern> or

  <Attribute> NOT LIKE <pattern>

- Pattern is a quoted string with

  %: any string;

  _: any character

- Example:

  Find the movies whose title contains the string "Bird".

  Return their title.

```
SELECT title
FROM movie
WHERE title LIKE '%Bird%';
```

NULL Values & Comparison

- Tuples in SQL relations can have NULL as a value
- WHERE … IS NULL:

  Check if an attribute value equals to NULL.

  Example: Find the movie stars where the year of birth is missing.

```
SELECT *
FROM Star
WHERE starborn IS NULL;
```

- WHERE … IS NOT NULL.

  Check if an attribute does not have a NULL value.

  Example:

```
SELECT *
FROM Star
WHERE starborn IS NOT NULL;
```

RMIT Classification: Trusted

DISTINCT: Removing Duplicates

- What are the distinct types of movies?

```
SELECT DISTINCT mvtype
FROM movie;
```

- Each different mvtype is listed only once.

ORDER BY: Ordering Output

- By default ORDER BY sort output into asending order. But can use ORDERY BY … DESC to sort into descending order.
- Example:

```
SELECT mvnumb, mvtitle, yrmde
FROM movie
ORDER BY yrmde DESC;
```

Aggregation Queries

- SUM, AVG, COUNT, MIN, and MAX can be applied to a column in a SELECT clause to produce that aggregation on the column.
- Also, COUNT(*) counts the number of tuples.
- Overall aggregation: Aggregate all tuples in a table.

Example: What's the most recent production year of movies?

```
SELECT MAX(yrmde)
FROM Movie;
```

Exercise

- Find the title of movies that are either a 'comedy' or a 'drama'
  - Modify the query to handle variations in case
  - Rewrite this query to use the IN clause
  - Find the title of all other types of movies
  - How many movies fall into these categories?
- Find the title of movies that are either a comedy or drama made between 1971 and 1979
  - Rewrite this to use between operator
- How many movie categories are there in the database?
- How many directors are there in the database?
- What was the average age of directors when they died?
  - Rewrite this query to use other AGGREGATE FUNCTIONS

- Q/A session